```
# The following imports are typically used for working with transformer models and PEFT.
# They are included here to provide context for the explanations.
import torch
from transformers import AutoModelForSequenceClassification, AutoTokenizer
from datasets import load_dataset
from peft import LoraConfig, get_peft_model

"""
Explanation Requirement:

Why Transformer Models Require Large Computational Resources:

1.  **Massive Number of Parameters**: State-of-the-art transformer models (like BERT, GPT, T5) often have hundreds of

2.  **Self-Attention Mechanism**: The core of transformer models is the self-attention mechanism, which calculates att

3.  **Large Pre-training Datasets**: To achieve their impressive performance, transformer models are typically pre-tra

4.  **Deep Architectures**: Transformers are deep neural networks with many layers (e.g., 12, 24, or even 96 layers).

5.  **Memory Footprint**: Due to the large number of parameters and intermediate activations generated during computat

Why PEFT Methods (Parameter-Efficient Fine-Tuning) Are Useful:

PEFT methods are designed to significantly reduce the computational and memory costs associated with fine-tuning large

1.  **Reduced Training Costs**: Instead of fine-tuning all parameters of a large pre-trained model, PEFT methods only

2.  **Lower Memory Consumption**: By only training a fraction of the parameters, PEFT methods require less memory for

3.  **Avoid Catastrophic Forgetting**: Fine-tuning the entire pre-trained model on a small, task-specific dataset can

4.  **Improved Generalization**: In some cases, focusing on a smaller set of parameters can lead to better generalizat

5.  **Easier Deployment and Storage**: Storing and deploying a full fine-tuned model for every downstream task can be

6.  **Examples of PEFT Methods**: Common PEFT techniques include LoRA (Low-Rank Adaptation), Prefix-tuning, Prompt-tun
"""
```

```
'\nExplanation Requirement:\n\nWhy Transformer Models Require Large Computational Resources:\n\n1.  **Massive Number
of Parameters**: State-of-the-art transformer models (like BERT, GPT, T5) often have hundreds of millions, or even bi
llions, of parameters. Each parameter requires memory for storage and computational operations (multiplications, addi
tions) during training and inference.\n\n2.  **Self-Attention Mechanism**: The core of transformer models is the self
-attention mechanism, which calculates attention scores between every pair of tokens in a sequence. This operation ha
s a quadratic complexity (O(N^2)) with respect to the sequence length (N). For long sequences, this quickly becomes a
computational bottleneck, both in terms of memory and processing time.\n\n3.  **Large Pre-training Datasets**: To ach
ieve their impressive performance, transformer models are typically pre-trained on enormous datasets (e.g., Common Cr
awl, Wikipedia). Training on such vast amounts of data requires …'
```

```
model_name = "distilbert-base-uncased"

tokenizer = AutoTokenizer.from_pretrained(model_name)

model = AutoModelForSequenceClassification.from_pretrained(
    model_name,
    num_labels=2
)
```

```
/usr/local/lib/python3.12/dist-packages/huggingface_hub/utils/_auth.py:94: UserWarning:
The secret `HF_TOKEN` does not exist in your Colab secrets.
To authenticate with the Hugging Face Hub, create a token in your settings tab (https://huggingface.co/settings/tokens
You will be able to reuse this secret in all of your notebooks.
Please note that authentication is recommended but still optional to access public models or datasets.
  warnings.warn(
```

config.json: 100% 483/483 [00:00<00:00, 37.2kB/s]

```
Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_TOKEN to enable higher rate limits ar
WARNING:huggingface_hub.utils._http:Warning: You are sending unauthenticated requests to the HF Hub. Please set a HF_T
```

tokenizer_config.json: 100% 48.0/48.0 [00:00<00:00, 3.74kB/s]

vocab.txt: 232k/? [00:00<00:00, 4.09MB/s]

tokenizer.json: 466k/? [00:00<00:00, 10.9MB/s]

model.safetensors: 100% 268M/268M [00:02<00:00, 223MB/s]

Loading weights: 100% 100/100 [00:00<00:00, 319.27it/s, Materializing param=distilbert.transformer.layer.5.sa_layer_norm.weight]

```
DistilBertForSequenceClassification LOAD REPORT from: distilbert-base-uncased
Key                      | Status     |
-------------------------+------------+-
vocab_transform.weight   | UNEXPECTED |
vocab_transform.bias     | UNEXPECTED |
vocab_projector.bias     | UNEXPECTED |
vocab_layer_norm.bias    | UNEXPECTED |
vocab_layer_norm.weight  | UNEXPECTED |
pre_classifier.bias      | MISSING    |
classifier.weight        | MISSING    |
pre_classifier.weight    | MISSING    |
classifier.bias          | MISSING    |

Notes:
- UNEXPECTED    :can be ignored when loading from different task/architecture; not ok if you expect identical arch.
- MISSING   :those params were newly initialized because missing from the checkpoint. Consider training on your downst
```

```
dataset = load_dataset("imdb")

def tokenize_function(example):
    # The 'text' field in the IMDb dataset is often a list, so we access the first element.
    return tokenizer(example["text"], truncation=True, padding="max_length")

tokenized_dataset = dataset.map(tokenize_function, batched=True)

train_dataset = tokenized_dataset["train"].shuffle(seed=42).select(range(500))
test_dataset = tokenized_dataset["test"].shuffle(seed=42).select(range(200))
```

README.md: 7.81k/? [00:00<00:00, 162kB/s]

plain_text/train-00000-of-00001.parquet: 100% 21.0M/21.0M [00:00<00:00, 106MB/s]

plain_text/test-00000-of-00001.parquet: 100% 20.5M/20.5M [00:00<00:00, 102MB/s]

plain_text/unsupervised-00000-of-00001.p(…): 100% 42.0M/42.0M [00:01<00:00, 110MB/s]

Generating train split: 100% 25000/25000 [00:00<00:00, 35734.51 examples/s]

Generating test split: 100% 25000/25000 [00:00<00:00, 36409.24 examples/s]

Generating unsupervised split: 100% 50000/50000 [00:01<00:00, 41079.50 examples/s]

Map: 100% 25000/25000 [00:48<00:00, 821.68 examples/s]

Map: 100% 25000/25000 [00:34<00:00, 703.90 examples/s]

Map: 100% 50000/50000 [01:10<00:00, 749.93 examples/s]

```
lora_config = LoraConfig(
    r=8,
    lora_alpha=16,
    target_modules=["q_lin", "v_lin"],
    lora_dropout=0.1,
    bias="none",
    task_type="SEQ_CLS"
```

```
    task_type="SEQ_CLS"
)
```

```
peft_model = get_peft_model(model, lora_config)

peft_model.print_trainable_parameters()
```

```
WARNING:torchao.kernel.intmm:Warning: Detected no triton, on systems without Triton certain kernels will not work
trainable params: 739,586 || all params: 67,694,596 || trainable%: 1.0925
```

```
from transformers import TrainingArguments, Trainer

training_args = TrainingArguments(
    output_dir="./results",
    learning_rate=2e-4,
    per_device_train_batch_size=8,
    num_train_epochs=1,
    logging_steps=10,
    save_strategy="no",
    report_to=[] # Disable reporting to services like Weights & Biases
)
```

```
trainer = Trainer(
    model=peft_model,
    args=training_args,
    train_dataset=train_dataset
)

trainer.train()
```

```
/usr/local/lib/python3.12/dist-packages/torch/utils/data/dataloader.py:775: UserWarning: 'pin_memory' argument is set
  super().__init__(loader)
```

[63/63 22:59, Epoch 1/1]

Step	Training Loss
10	0.707690
20	0.656838
30	0.671451
40	0.632742
50	0.632311
60	0.610926

```
TrainOutput(global_step=63, training_loss=0.6503299153040326, metrics={'train_runtime': 1419.2963,
'train_samples_per_second': 0.352, 'train_steps_per_second': 0.044, 'total_flos': 67369703424000.0, 'train_loss':
0.6503299153040326, 'epoch': 1.0})
```